Universidad del Valle de Guatemala
Programación Orientada a Objetos
Segundo ciclo, 2026

Laboratorio 3 - Herencia

Lourdes Lemus
26305
15/10/26

Análisis del laboratorio 3 de herencia

TecniHogar requiere una aplicación de consola en Java para cotizar y registrar servicios, organizar jornadas y controlar visitas e ingresos. El análisis establece sus requisitos, estados y clases, distribuyendo mediante herencia y polimorfismo las decisiones propias de cada categoría.

1. Requisitos funcionales

1) Información que debe conservar el sistema

Elemento	Información
Orden de servicio	Código único, nombre del cliente, dirección, horas estimadas decimales, zona, estado y cantidad de visitas realizadas. La categoría se identifica mediante la clase concreta.
Instalación	Cantidad de puntos a instalar, además de los datos comunes.
Instalación eléctrica certificada	Valor declarado de materiales, además de los datos de instalación.
Mantenimiento de aire acondicionado	Capacidad del equipo en BTU, además de los datos comunes.
Cuadrilla	Código único, capacidad máxima de horas por jornada y certificación en refrigeración.
Jornada	Cuadrilla, estado, órdenes asignadas y resultado de cada visita de esa jornada. Se propone un código único para seleccionarla en el menú.
Ingreso	Orden que originó el cobro, concepto y monto. Los conceptos son registro y visita adicional.

Toda la información se conserva únicamente en memoria. No se requieren archivos de datos, base de datos ni interfaz gráfica.

2) Operaciones y condiciones de validez

Operación	Condiciones y efecto
Cotizar un servicio	Solicitar la categoría y sus datos; validar y mostrar costo y días hábiles. No exigir código de registro, guardar la orden ni generar ingresos.
Registrar una orden	Validar los datos y un código no vacío y no repetido. Guardar la orden pendiente, con cero visitas, y registrar una sola vez el monto completo de su servicio.
Registrar una cuadrilla	Exigir código no vacío y único, capacidad positiva e indicación de certificación.
Crear una jornada	Seleccionar una cuadrilla existente que no tenga otra jornada en preparación o iniciada. Crear la jornada en preparación y sin órdenes.

Operación	Condiciones y efecto
Asignar una orden	Exigir jornada en preparación y orden registrada pendiente. Debe tener visitas disponibles, no estar asignada a otra jornada, ser compatible con la cuadrilla y caber en la capacidad restante.
Retirar una orden	Exigir jornada en preparación y que la orden pertenezca a ella. Eliminar la asignación, devolver la orden a pendiente y liberar sus horas. No generar visita ni cobro.
Iniciar una jornada	Exigir jornada en preparación y no vacía. Pasar todas sus órdenes a atención y aumentar sus visitas en uno. Cobrar Q40 por cada orden cuya visita iniciada sea posterior a la primera.
Registrar resultado	Exigir jornada iniciada, orden perteneciente a ella y resultado aún no registrado. Aceptar completada o visita fallida y actualizar la orden conforme a sus visitas disponibles.
Cerrar una jornada	Cerrar automáticamente al registrarse el resultado de todas sus órdenes. La cuadrilla queda libre para una nueva jornada.
Consultar órdenes y cuadrillas	Mostrar los datos registrados, incluidos el estado y las visitas de las órdenes y la capacidad y certificación de las cuadrillas.
Generar reportes	Mostrar la cantidad de órdenes por categoría y estado, los ingresos totales separados por registro y visitas adicionales, los ingresos por categoría y las jornadas sin cerrar con su cuadrilla, horas asignadas y capacidad disponible.
Usar el menú	Permitir las operaciones anteriores repetidamente hasta elegir salir. Una entrada con formato incorrecto no debe terminar el programa.

3) Reglas de cobro y validación

Las horas estimadas y la capacidad de la cuadrilla deben ser positivas y finitas. Los puntos deben ser enteros positivos; el valor declarado de materiales debe ser positivo. Los códigos se validan sin espacios externos y no pueden quedar vacíos. Los BTU deben ser enteros entre 9,000 y 60,000, incluidos ambos límites. La zona solo puede ser metropolitana o periférica.

Diagnóstico: admite hasta 1.5 horas estimadas. Cuesta Q150.00 en zona metropolitana y Q225.00 en periférica, independientemente de las horas.

Instalación: la base es Q100.00 en zona metropolitana y Q175.00 en periférica. Las horas por puntos son puntos × 0.75. Se toma el mayor valor entre horas estimadas y horas por puntos, se redondea al entero superior y se multiplica por Q90.00; el resultado se suma a la base. El redondeo se aplica una sola vez al mayor valor.

Instalación eléctrica certificada: utiliza el costo de instalación y agrega Q120.00 por certificado y el 5 % del valor declarado de los materiales por garantía.

Mantenimiento de aire acondicionado: solo se admite en zona metropolitana. Cuesta la tarifa base de esa zona, más Q110.00 por cada hora estimada y Q80.00 por gas refrigerante. Las horas estimadas se usan sin redondearlas al entero superior. Se interpreta la tarifa base mencionada en el

caso como Q100.00 metropolitana y Q175.00 periférica; la restricción de zona hace aplicable únicamente Q100.00 al mantenimiento.

Para la capacidad de una jornada se suman las horas estimadas originales, sin redondeo ni sustitución por horas facturables. La suma puede igualar la capacidad máxima, pero no superarla. Solo el mantenimiento exige una cuadrilla certificada en refrigeración; la instalación eléctrica certificada no tiene una restricción adicional de cuadrilla en el enunciado.

Los montos se manejan con BigDecimal, se redondean a centavos con HALF_UP y se presentan con dos decimales. El costo del servicio se redondea al finalizar su cálculo. Los valores y operaciones inválidos se rechazan con un mensaje claro y sin modificar datos, estados, visitas, asignaciones o ingresos. Primero se validan todas las condiciones y después se aplican los cambios de la operación.

4) Ingresos reportes y datos iniciales

El ingreso de registro se genera únicamente cuando se acepta y registra una nueva orden durante el uso del programa. Cada visita adicional a la primera genera Q40.00 únicamente al iniciar su jornada. Una cotización, una asignación, un retiro o una operación rechazada no generan ingresos. Una visita fallida o una cancelación no producen devolución de cobros, pues el caso no establece reembolsos.

El reporte de ingresos muestra el total, el subtotal por registro, el subtotal por visitas adicionales y el total por categoría sumando ambos conceptos. Se calcula desde los cobros efectivamente registrados, no desde el costo de todas las órdenes existentes. El reporte de jornadas sin cerrar incluye las jornadas en preparación e iniciadas, su cuadrilla, la suma de horas estimadas asignadas y la capacidad disponible. Esta capacidad equivale al máximo de la cuadrilla menos las horas asignadas.

Al iniciar se cargan al menos dos órdenes pendientes de cada una de las cuatro categorías y al menos tres cuadrillas: una certificada en refrigeración y otra no certificada como mínimo. Todas las órdenes comienzan con cero visitas y sin ingresos registrados. La carga inicial evita el cobro de registro. Los datos permiten observar las reglas de cobro, ambas zonas donde corresponda y una instalación cuyas horas por puntos superen las estimadas, con fracción que requiera redondeo.

2. Decisiones que dependen de la categoría

Cada clase concreta responde por su costo, tiempo de atención, máximo de visitas, identificación de categoría y validaciones específicas. La compatibilidad general se implementa en OrdenServicio y el mantenimiento la sobrescribe. Las jornadas invocan estos comportamientos sin usar instanceof ni comparaciones de categoría para decidir reglas.

Clase	Decisiones y justificación
Diagnostico	Valida el máximo de 1.5 horas. Calcula Q150.00 o Q225.00 según zona; estima 1 o 2 días hábiles, respectivamente; permite 3 visitas. Estas reglas corresponden exclusivamente al diagnóstico.
Instalacion	Valida puntos positivos. Calcula base, máximo de horas y redondeo por fracción; estima 3 días metropolitanos o 5 periféricos y permite 3 visitas. Define el comportamiento reutilizable de una instalación.
InstalacionElectrica Certificada	Valida materiales positivos. Reutiliza el costo de instalación y agrega certificado y garantía; reutiliza su tiempo y suma 2 días, por lo que estima 5 o 7 días. Permite 2 visitas. Solo modifica las reglas que la distinguen de su clase padre.

Clase	Decisiones y justificación
Mantenimiento AireAcondicionado	Valida BTU y zona metropolitana. Calcula base, horas sin redondear y gas; estima 2 días y permite 1 visita. Sobrescribe compatibilidad para exigir refrigeración, porque esta restricción pertenece al servicio.
OrdenServicio	Acepta por defecto cualquier cuadrilla. Define las transiciones comunes y consulta polimórficamente el máximo de visitas para decidir si una visita fallida deja pendiente o cancelada la orden.

El cargo adicional de Q40.00, la unicidad de códigos, el control de capacidad y el cierre de jornadas son reglas comunes: no se duplican en las subclases. Jornada controla capacidad y pertenencia; SistemaTecniHogar coordina registros e ingresos. El límite de visitas y la compatibilidad se consultan a la orden.

3. Estados y transiciones

1) Estados de una orden

Estado inicial	Operación o condición	Estado final
Sin registro	Aceptar y registrar la orden válida.	PENDIENTE
PENDIENTE	Asignarla válidamente a una jornada en preparación.	ASIGNADA
ASIGNADA	Retirarla durante la preparación de su jornada.	PENDIENTE
ASIGNADA	Iniciar su jornada y contar una visita.	EN_ATENCION
EN_ATENCION	Registrar resultado completada, incluso en la última visita permitida.	COMPLETADA
EN_ATENCION	Registrar visita fallida con visitas realizadas menores que el máximo.	PENDIENTE
EN_ATENCION	Registrar visita fallida con visitas realizadas iguales al máximo.	CANCELADA

Sin registro representa la situación previa al alta, no un valor de EstadoOrden. Una cotización no incorpora una orden al sistema. COMPLETADA y CANCELADA son estados finales. Una orden no se cancela por completar su última visita: se cancela si esa visita falla. El máximo es 3 para diagnóstico e instalación, 2 para instalación eléctrica certificada y 1 para mantenimiento.

El estado ASIGNADA impide que una orden pendiente de atención pertenezca simultáneamente a dos jornadas activas. Tras una visita fallida puede seguir apareciendo en el historial de la jornada anterior, pero esa pertenencia histórica no constituye una asignación activa. Cada resultado corresponde a una visita de una jornada específica.

2) Estados de una jornada

Estado inicial	Operación o condición	Estado final
Sin crear	Crear la jornada para una cuadrilla disponible.	EN_PREPARACION
EN_PREPARACION	Asignar o retirar órdenes válidamente.	EN_PREPARACION
EN_PREPARACION	Iniciar la jornada con al menos una orden.	INICIADA
INICIADA	Registrar un resultado cuando todavía faltan otros.	INICIADA
INICIADA	Registrar el último resultado pendiente.	CERRADA

Una jornada vacía no inicia. Después de iniciar no se asignan ni retiran órdenes. Una jornada cerrada no se reinicia ni admite resultados adicionales. La cuadrilla conserva su ocupación hasta cerrar la jornada, aunque algunas órdenes ya tengan resultado. El número de visitas aumenta al inicio, nunca al asignar o registrar el resultado.

4. Clases responsabilidades atributos y métodos

La visibilidad se indica con + para público, - para privado y # para protegido. Los métodos incluyen sus parámetros y tipo de retorno; los constructores no tienen tipo de retorno.

1) Tipos enumerados

Tipo	Valores y propósito
Zona	METROPOLITANA, PERIFERICA. Restringe las zonas válidas.
EstadoOrden	PENDIENTE, ASIGNADA, EN_ATENCION, COMPLETADA, CANCELADA. Describe el avance de la orden.
EstadoJornada	EN_PREPARACION, INICIADA, CERRADA. Controla las operaciones de la jornada.
ResultadoVisita	COMPLETADA, FALLIDA. Representa el resultado de una visita.
ConceptoIngreso	REGISTRO, VISITA_ADICIONAL. Separa los cobros para reportarlos.

2) OrdenServicio abstracta

Responsabilidad: conservar los datos comunes, proteger el estado y definir el contrato polimórfico. Atributos privados: codigo: String, identificador asignado al registrar; cliente: String y direccion: String, datos de atención; horasEstimadas: double, duración prevista; zona: Zona, ubicación; estado: EstadoOrden, situación actual; visitasRealizadas: int, cantidad de visitas iniciadas.

Métodos y constructores	Propósito
# OrdenServicio(cliente: String, direccion: String, horasEstimadas: double, zona: Zona)	Validar los datos comunes e iniciar codigo en null, estado en PENDIENTE y visitasRealizadas en 0. El objeto todavía no pertenece al registro.
+ calcularCosto(): BigDecimal [abstracto]	Devolver el costo completo del servicio según sus reglas.
# calcularTarifaBase(): BigDecimal	Devolver Q100.00 para zona metropolitana o Q175.00 para periférica. Instalacion y MantenimientoAireAcondicionado reutilizan esta regla común, sin duplicar la selección por zona.
+ estimarDiasAtencion(): int [abstracto]	Devolver los días hábiles estimados.
+ obtenerMaximoVisitas(): int [abstracto]	Devolver el número de visitas permitido.
+ obtenerCategoria(): String [abstracto]	Devolver el nombre de categoría para consultas y agrupación.
+ esCompatibleCon(cuadrilla: Cuadrilla): boolean	Devolver true para una cuadrilla existente; mantenimiento sobrescribe esta regla.
+ asignarCodigo(codigo: String): void	Asignar una sola vez el código no vacío; la unicidad global la valida el sistema.
+ marcarAsignada(): void	Exigir PENDIENTE y visitas disponibles; pasar a ASIGNADA.
+ retirarAsignacion(): void	Exigir ASIGNADA y pasar a PENDIENTE.
+ iniciarVisita(): void	Exigir ASIGNADA y visitas disponibles; incrementar visitas y pasar a EN_ATENCION.
+ registrarResultado(resultado: ResultadoVisita): void	Exigir EN_ATENCION y resultado válido. Completar, volver a pendiente o cancelar según el resultado y el máximo de visitas.
+ getCodigo(): String; + getCliente(): String; + getDireccion(): String	Consultar identidad y datos del cliente.
+ getHorasEstimadas(): double; + getZona(): Zona; + getEstado(): EstadoOrden; + getVisitasRealizadas(): int	Consultar duración, ubicación y progreso sin permitir cambios directos.

Los atributos comunes no se repiten en las subclases. No hay setters públicos para código, zona, horas, estado o visitas. El objeto temporal usado para cotizar no recibe código ni se añade a las colecciones. Si el cliente no acepta, se descarta; si acepta, SistemaTecniHogar valida y asigna el código antes de incorporarlo y cobrarlo. El método protegido calcularTarifaBase() centraliza la base compartida por instalación y mantenimiento; cada subclase conserva su fórmula de costo.

3) Diagnostico

Hereda de OrdenServicio y no agrega atributos. Constructor público Diagnostico(cliente: String, direccion: String, horasEstimadas: double, zona: Zona): llama a super con los datos comunes y rechaza horas mayores de 1.5.

Métodos públicos sobrescritos: calcularCosto(): BigDecimal, devuelve Q150.00 o Q225.00 según zona; estimarDiasAtencion(): int, devuelve 1 o 2; obtenerMaximoVisitas(): int, devuelve 3; obtenerCategoria(): String, devuelve Diagnóstico. Hereda la compatibilidad general.

4) Instalacion

Hereda de OrdenServicio. Atributo privado puntos: int, cantidad de puntos utilizados para calcular las horas por puntos. Constructor público Instalacion(cliente: String, direccion: String, horasEstimadas: double, zona: Zona, puntos: int): llama a super y valida puntos positivos.

Métodos públicos sobrescritos: calcularCosto(): BigDecimal, utiliza calcularTarifaBase() y suma Q90.00 por el máximo de horas redondeado al entero superior; estimarDiasAtencion(): int, devuelve 3 o 5 según zona; obtenerMaximoVisitas(): int, devuelve 3; obtenerCategoria(): String, devuelve Instalación. Método público getPuntos(): int, consulta los puntos. Hereda la compatibilidad general.

5) InstalacionElectricaCertificada

Hereda de Instalacion. Atributo privado valorMateriales: BigDecimal, importe declarado para la garantía. Constructor público InstalacionElectricaCertificada(cliente: String, direccion: String, horasEstimadas: double, zona: Zona, puntos: int, valorMateriales: BigDecimal): llama a super con los datos de instalación y valida materiales positivos.

Métodos públicos sobrescritos: calcularCosto(): BigDecimal, utiliza super.calcularCosto() y agrega Q120.00 más 5 % de materiales; estimarDiasAtencion(): int, devuelve super.estimarDiasAtencion() + 2; obtenerMaximoVisitas(): int, devuelve 2; obtenerCategoria(): String, devuelve Instalación eléctrica certificada. Método público getValorMateriales(): BigDecimal, consulta el importe. Hereda los puntos y la compatibilidad general.

6) MantenimientoAireAcondicionado

Hereda de OrdenServicio. Atributo privado btu: int, capacidad del equipo. Constructor público MantenimientoAireAcondicionado(cliente: String, direccion: String, horasEstimadas: double, zona: Zona, btu: int): llama a super y valida el rango inclusivo de BTU y la zona metropolitana.

Métodos públicos sobrescritos: calcularCosto(): BigDecimal, utiliza calcularTarifaBase() y suma Q110.00 por hora estimada más Q80.00; estimarDiasAtencion(): int, devuelve 2; obtenerMaximoVisitas(): int, devuelve 1; obtenerCategoria(): String, devuelve Mantenimiento de aire acondicionado; esCompatibleCon(cuadrilla: Cuadrilla): boolean, exige certificación en refrigeración. Método público getBtu(): int, consulta la capacidad.

7) Cuadrilla

Responsabilidad: conservar sus datos y capacidad. Atributos privados: codigo: String, identificador único; capacidadHoras: double, máximo por jornada; certificadaRefrigeracion: boolean, habilitación para mantenimiento. Constructor público Cuadrilla(codigo: String, capacidadHoras: double, certificadaRefrigeracion: boolean): valida código y capacidad; el sistema comprueba unicidad.

Métodos públicos: getCodigo(): String, consulta el código; getCapacidadHoras(): double, consulta el máximo; isCertificadaRefrigeracion(): boolean, consulta la certificación. No conserva un indicador de disponibilidad: el sistema la obtiene de las jornadas sin cerrar, evitando duplicar ese estado.

8) Jornada

Responsabilidad: administrar la asignación, capacidad, inicio y resultados. Atributos privados: codigo: String, identificador; cuadrilla: Cuadrilla, equipo responsable; estado: EstadoJornada, fase de trabajo; ordenes: List<OrdenServicio>, servicios de la jornada; resultados: Map<String, ResultadoVisita>, resultados indexados por código de orden. Las horas y la capacidad disponible se calculan, no se almacenan duplicadas.

Métodos y constructores	Propósito
+ Jornada(codigo: String, cuadrilla: Cuadrilla)	Exigir código y cuadrilla válidos; iniciar EN_PREPARACION con colecciones vacías. El sistema valida unicidad y disponibilidad.
+ asignar(orden: OrdenServicio): void	Validar preparación, orden registrada y pendiente, visitas, no duplicación, capacidad y orden.esCompatibleCon(cuadrilla); luego añadir y marcar asignada.
+ retirar(codigoOrden: String): void	Validar preparación y pertenencia; quitar la orden y devolverla a pendiente.
+ validarInicio(): void	Comprobar preparación, jornada no vacía y validez de todas las órdenes antes de modificar alguna.
+ iniciar(): void	Tras validar, iniciar las visitas de todas las órdenes y cambiar a INICIADA. El sistema coordina los cobros asociados.
+ registrarResultado(codigoOrden: String, resultado: ResultadoVisita): void	Validar jornada iniciada, pertenencia y ausencia de resultado previo; actualizar la orden, guardar resultado y cerrar si todas tienen resultado.
+ calcularHorasAsignadas(): double	Sumar horas estimadas de las órdenes.
+ calcularCapacidadDisponible(): double	Restar horas asignadas a la capacidad máxima de la cuadrilla.
+ getCodigo(): String; + getCuadrilla(): Cuadrilla; + getEstado(): EstadoJornada	Consultar identidad, equipo y estado.
+ getOrdenes(): List<OrdenServicio>; + getResultados(): Map<String, ResultadoVisita>	Consultar copias o vistas de solo lectura de las órdenes y resultados.

Una jornada referencia una sola cuadrilla y de cero a muchas órdenes. Una cuadrilla puede tener varias jornadas históricas, pero como máximo una sin cerrar. Una orden puede figurar en varias jornadas históricas por reprogramación, pero solo tener una asignación activa. Los resultados permanecen asociados a su jornada aunque la orden se atienda después en otra.

9) Ingreso

Responsabilidad: representar un cobro realizado. Atributos privados e inmutables: orden: OrdenServicio, origen del ingreso y de su categoría; concepto: ConceptoIngreso, tipo de cobro; monto: BigDecimal, valor cobrado. No se duplica el nombre de categoría: se obtiene de la orden.

Constructor público Ingreso(orden: OrdenServicio, concepto: ConceptoIngreso, monto: BigDecimal): exige orden registrada, concepto válido y monto positivo. Métodos públicos getOrden(): OrdenServicio, getConcepto(): ConceptoIngreso y getMonto(): BigDecimal: consultan los datos del cobro. La clase no ofrece setters.

10) SistemaTecniHogar

Responsabilidad: coordinar las operaciones completas y los reportes. Atributos privados: ordenes: Map<String, OrdenServicio>, cuadrillas: Map<String, Cuadrilla> y jornadas: Map<String, Jornada>, registros por código; ingresos: List<Ingreso>, cobros efectuados. Constructor público SistemaTecniHogar(): crea las colecciones y ejecuta la carga inicial sin ingresos.

Métodos y constructores	Propósito
+ cotizar(orden: OrdenServicio): String	Recibir un objeto temporal válido sin código y devolver costo y días calculados polimórficamente; no guardarlo ni cobrarlo.
+ registrarOrden(codigo: String, orden: OrdenServicio): void	Validar código único y objeto no registrado; calcular el monto, asignar código, incorporar la orden y registrar su ingreso una sola vez.
+ registrarCuadrilla(cuadrilla: Cuadrilla): void	Validar código único e incorporar la cuadrilla.
+ crearJornada(codigoJornada: String, codigoCuadrilla: String): void	Validar código único, cuadrilla existente y ausencia de otra jornada sin cerrar; guardar la nueva jornada.
+ asignarOrden(codigoJornada: String, codigoOrden: String): void	Resolver referencias y delegar la asignación válida a Jornada.
+ retirarOrden(codigoJornada: String, codigoOrden: String): void	Resolver referencias y delegar el retiro sin ingresos.
+ iniciarJornada(codigoJornada: String): void	Validar todas las órdenes; preparar Q40.00 para cada visita posterior a la primera; iniciar la jornada y registrar esos cobros como una sola operación válida.
+ registrarResultado(codigoJornada: String, codigoOrden: String, resultado: ResultadoVisita): void	Localizar la jornada y delegar el resultado de esa visita; no generar cobros por registrar el resultado.
+ consultarOrdenes(): List<OrdenServicio>; + consultarCuadrillas(): List<Cuadrilla>; + consultarJornadas(): List<Jornada>	Devolver copias o vistas de solo lectura para consulta y selección en el menú.
+ generarReporteOrdenes(): String	Agrupar dinámicamente por categoría y estado e incluir sus totales.

Métodos y constructores	Propósito
+ generarReporteIngresos(): String	Sumar cobros por concepto, categoría y total general.
+ generarReporteJornadas(): String	Mostrar jornadas sin cerrar con cuadrilla, horas y capacidad disponible.
- cargarDatosIniciales(): void	Crear al menos ocho órdenes pendientes y tres cuadrillas. Asignar códigos únicos e incorporarlas sin agregar cobros a ingresos.

El inicio valida la operación completa antes de cambiar cualquier orden. Los importes se determinan con el número de visitas anterior al inicio: si ya hubo al menos una, corresponde Q40.00. Una nueva solicitud de inicio sobre una jornada iniciada se rechaza antes de contar visitas o cobrar otra vez. Los reportes iteran los registros sin decidir reglas por tipo concreto.

11) Aplicacion

Responsabilidad: interacción por consola. Atributos privados: sistema: SistemaTecniHogar, acceso a la lógica; entrada: Scanner, lectura del teclado. Constructor público Aplicacion(): crea ambos objetos. Métodos: + static main(args: String[]): void, crea Aplicacion y ejecuta el menú; + ejecutarMenu(): void, repite las opciones hasta salir; - ejecutarOpcion(opcion: int): void, solicita los datos y llama al sistema; - leerTexto(mensaje: String): String, lee texto; - leerEntero(mensaje: String): int, - leerDecimal(mensaje: String): double y - leerMonto(mensaje: String): BigDecimal, repiten la lectura ante formato inválido; - leerZona(): Zona y - leerResultado(): ResultadoVisita, restringen las opciones; - leerServicio(): OrdenServicio, solicita categoría y datos y construye el objeto concreto sin registrarlo.

El menú selecciona la clase concreta al crear el servicio. Después, costo, tiempo, compatibilidad y límite de visitas se consultan por polimorfismo. Aplicacion captura los errores de entrada y las operaciones rechazadas, muestra su mensaje y continúa; no contiene las fórmulas de cobro ni modifica directamente estados o ingresos.

12) Inicialización y colaboración de constructores

Los constructores reciben los datos del servicio y los validan antes de que el objeto se incorpore al sistema. Cada constructor concreto llama primero al constructor de su clase padre mediante super(...). OrdenServicio inicializa la parte común; Instalacion valida e inicializa puntos; InstalacionElectricaCertificada reutiliza ambas inicializaciones y agrega valorMateriales. Diagnostico y MantenimientoAireAcondicionado completan sus validaciones propias después de inicializar la parte común. Un constructor rechazado no incorpora objetos ni cobros al sistema.

Las subclases usan los métodos de consulta heredados para acceder a los datos privados del padre. El código de orden se asigna al aceptar el registro porque una cotización no requiere identificador. Tanto el registro normal como la carga inicial dejan la orden pendiente y con cero visitas; únicamente el registro normal añade un ingreso. Jornada inicia sus propias colecciones vacías y recibe una cuadrilla previamente registrada.

5. Justificación de la relación entre instalaciones

InstalacionElectricaCertificada hereda de Instalacion porque representa una instalación con puntos y con la misma regla de horas facturables, base por zona y tarifa horaria. Agrega el valor de materiales y modifica costo, tiempo y máximo de visitas. La relación permite tratar ambos servicios como

instalaciones y como órdenes de servicio, respetando que sus políticas se consultan mediante métodos sobrescritos.

El uso de super.calcularCosto() evita duplicar la fórmula de instalación y super.estimarDiasAtencion() evita duplicar los plazos por zona. La subclase conserva los atributos y validaciones de instalación, agrega certificado y garantía y reduce su límite a dos visitas. La jornada no supone que toda instalación permite tres visitas: consulta obtenerMaximoVisitas() sobre la orden concreta.

Una alternativa sería que ambas clases heredaran directamente de OrdenServicio. Se ganaría independencia entre ambas categorías, útil si sus reglas dejaran de compartir significado. Se perdería la reutilización directa del comportamiento de instalación y habría que duplicar sus fórmulas o extraerlas a un componente compartido. Para las reglas del caso, la herencia desde Instalacion expresa la relación entre los servicios y evita duplicación sin añadir otro componente.

6. Incorporación de una nueva categoría

Para incorporar paneles solares o cámaras de seguridad se crea una subclase de OrdenServicio, o de Instalacion si realmente comparte las reglas de instalación. La nueva clase incorpora sus atributos y constructor, valida sus datos e implementa o sobrescribe calcularCosto(), estimarDiasAtencion(), obtenerMaximoVisitas(), obtenerCategoria() y esCompatibleCon(Cuadrilla) cuando exista una restricción de cuadrilla.

Se modifica leerServicio() de Aplicacion para ofrecer la categoría, solicitar sus datos y construirla. Solo si se desea incluirla en los datos precargados se amplía cargarDatosIniciales(). Si una nueva restricción requiere una característica de cuadrilla que aún no se representa, se amplían Cuadrilla, su constructor y su captura en el menú; la evaluación de esa restricción permanece en la nueva categoría.

No se modifican las fórmulas de las categorías existentes, los estados comunes ni la lógica de asignación y visitas de Jornada, siempre que la nueva categoría siga el ciclo definido en el caso. Jornada ya consulta compatibilidad y límite de visitas mediante el contrato de OrdenServicio. Los reportes tampoco requieren cambios: agrupan dinámicamente por obtenerCategoria() y suman ingresos existentes. El cargo común por visita adicional continúa registrándose al iniciar la jornada.